

CAPE COLLEGE OF ENGINEERING ALAPPUZHA

Department of _____

PCCSL 507 MACHINE LEARNING LAB

LABORATORY RECORD

Submitted by

Name : _____

Reg. No. : _____

Programme : _____

Semester / Year : _____

Academic Year : _____

Faculty In-Charge : _____

Signature : _____ Date : _____

INDEX

Sl. No.	Title of the Experiment	Date	Page No.	Signature
1	Simple Linear Regression (Least Squares & Gradient Descent)		5	
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				

EXPERIMENT NO. 1

Date: _____

Page No.: _____

TITLE

Simple Linear Regression using Least Squares and Gradient Descent

OBJECTIVE

To implement simple linear regression on housing.csv by least squares and gradient descent (X =TotalRooms, Y =MedHouseVal), and evaluate it with MSE, R-squared and the regression-line plot.

AIM

To predict MedHouseVal from TotalRooms with SLR (least squares and gradient descent, $\alpha = 0.01$, 100 epochs), and assess the fit using MSE and R^2 .

THEORY

Regression equation:

$$\hat{Y} = mX + b$$

Least-squares estimates:

$$m = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2} \quad b = \bar{Y} - m\bar{X}$$

Mean Squared Error cost:

$$J(m, c) = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Gradients:

$$\frac{\partial J}{\partial m} = -\frac{2}{n} \sum_{i=1}^n X_i (Y_i - \hat{Y}_i) \quad \frac{\partial J}{\partial b} = -\frac{2}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)$$

Gradient descent updates:

$$m := m - \alpha \frac{\partial J}{\partial m} \quad b := b - \alpha \frac{\partial J}{\partial b}$$

Goodness of fit:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad R^2 = 1 - \frac{\sum_{i=1}^n (Y_i - \hat{Y}_i)^2}{\sum_{i=1}^n (Y_i - \bar{Y})^2}$$

PROCEDURE

A) Least Squares: load housing.csv; take X =TotalRooms, Y =MedHouseVal; inspect missing, duplicate and invalid values; compute slope m and intercept b by least squares; form $\hat{Y} = mX + b$; predict the test set; report MSE and R^2 ; plot data points with the regression line.

B) Gradient Descent: load housing.csv (X =TotalRooms, Y =MedHouseVal); clean the data; 80/20 split; standardize X with train mean/std; init $m = c = 0$; set $\alpha = 0.01$, epochs=100; iterate predict $\rightarrow$ cost $J \rightarrow$ gradients $\rightarrow$ update m, b ; plot cost vs epochs; predict test data with final m, b ; report MSE and R^2 .

DATASET DESCRIPTION

Dataset Name: California Housing (housing.csv)

Source: Public census CSV file; prepared with NumPy only

Number of Samples: 20640 usable (0 missing, 118 duplicates, 0 invalid) Number of Features: 1 (TotalRooms)

Target / Class: MedHouseVal – median house value in units of \$100,000

Description: Train 16512 / test 4128 (80/20, seed 42); feature standardized.

PROGRAM

```

import csv
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

rooms, prices, miss = [], [], 0
with open("housing.csv", newline="") as f:
    for r in csv.DictReader(f):
        try:
            rooms.append(float(r["total_rooms"]))
            prices.append(float(r["median_house_value"]))
        except (ValueError, KeyError):
            miss += 1
X = np.array(rooms)
y = np.array(prices) / 1e5
print("Usable:", len(X), " Missing:", miss)

rng = np.random.default_rng(42)
order = np.arange(len(X))
rng.shuffle(order)
n_train = int(0.8 * len(X))
X_train, X_test = X[order[:n_train]], X[order[n_train:]]
y_train, y_test = y[order[:n_train]], y[order[n_train:]]
print("Train:", len(X_train), " Test:", len(X_test))

x_bar = X_train.mean()
y_bar = y_train.mean()
m = ((X_train - x_bar) * (y_train - y_bar)).sum()
m = m / ((X_train - x_bar) ** 2).sum()
b = y_bar - m * x_bar
print(f"LS slope m = {m:.6f}")
print(f"LS intercept b = {b:.4f}")

y_pred = m * X_test + b
mse = ((y_test - y_pred) ** 2).mean()
r2 = 1 - ((y_test - y_pred) ** 2).sum() / ((y_test - y_test.mean()) ** 2).sum()
print(f"LS: MSE={mse:.4f} R2={r2:.4f} RMSE={np.sqrt(mse):.4f}")

plt.figure(figsize=(7, 4.5))
plt.scatter(X_test, y_test, s=6, alpha=0.25, label="Test data")
xs = np.linspace(0, float(np.percentile(X, 99.5)), 200)
plt.plot(xs, m * xs + b, linewidth=2, label="Least Squares line")
plt.xlabel("TotalRooms (X)")
plt.ylabel("MedHouseVal ($100k) (Y)")
plt.title("SLR: TotalRooms vs Median House Value")
plt.legend()
plt.tight_layout()
plt.savefig("exp1_ls.png", dpi=150)

mu = X_train.mean()
sg = X_train.std()
Xs_train = (X_train - mu) / sg
m, b = 0.0, 0.0
n = len(Xs_train)
costs = []
for epoch in range(100):
    y_hat = m * Xs_train + b

```

```
err = y_train - y_hat
J = (err ** 2).mean()
costs.append(float(J))
dm = (-2 / n) * (Xs_train * err).sum()
db = (-2 / n) * err.sum()
m = m - 0.01 * dm
b = b - 0.01 * db
m_gd = m / sg
b_gd = b - m * mu / sg
print(f"GD slope m = {m_gd:.6f}")
print(f"GD intercept b = {b_gd:.4f}")
print(f"GD cost first={costs[0]:.4f} last={costs[-1]:.4f}")
y_pred = m_gd * X_test + b_gd
mse = ((y_test - y_pred) ** 2).mean()
r2 = 1 - ((y_test - y_pred) ** 2).sum() / ((y_test - y_test.mean()) ** 2).sum()
print(f"GD: MSE={mse:.4f} R2={r2:.4f} RMSE={np.sqrt(mse):.4f}")
plt.figure(figsize=(6, 3.8))
plt.plot(range(1, 101), costs)
plt.xlabel("Epoch")
plt.ylabel("Cost J (MSE)")
plt.title("Gradient Descent: Cost vs Epochs")
plt.tight_layout()
plt.savefig("exp1_cost.png", dpi=150)
```

OUTPUT

Usable: 20640 Missing: 0
Train: 16512 Test: 4128
LS slope $m = 0.000072$
LS intercept $b = 1.8840$
LS: MSE=1.3236 R2=0.0167 RMSE=1.1505
GD slope $m = 0.000062$
GD intercept $b = 1.6341$
GD cost first=5.6208 last=1.3827
GD: MSE=1.3885 R2=-0.0315 RMSE=1.1784

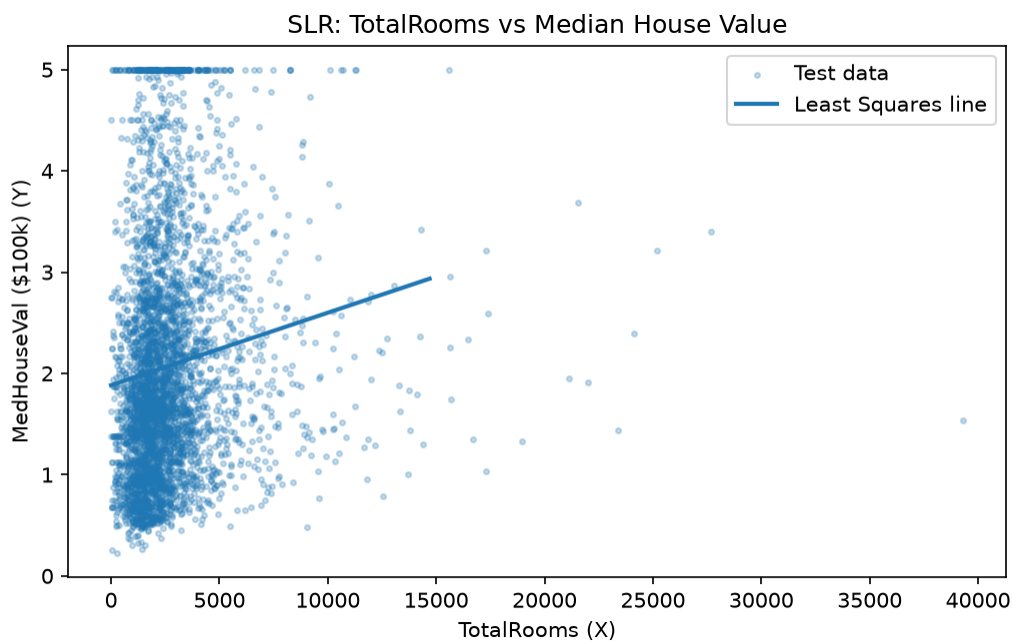


Fig. 1: Test data with the least-squares regression line.

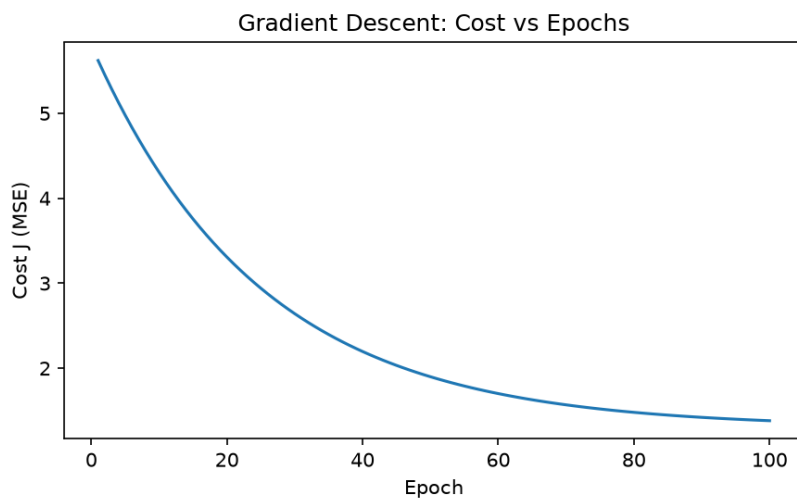


Fig. 2: Gradient-descent cost vs epochs ($\alpha = 0.01$, 100 epochs).

RESULT

The above program was implemented and executed successfully using Python, and the corresponding output was obtained and verified. The results of the least squares and gradient descent regression were analyzed and found to be satisfactory, thereby achieving the desired objective of the experiment.

Faculty Signature: _____

Date: _____